

ImageSharp

ImageSharp is the high-performance part of the Six Labors stack you reach for when you need to load, inspect, process, and save images entirely in managed .NET code. It gives you one consistent image model whether you are building a thumbnail service, a photo workflow, a web upload pipeline, or a lower-level imaging tool.

This section is written as a guided set of articles rather than a flat feature list. Start with [Getting Started](#) if you are new to the library, then branch into loading, processing, formats, or lower-level pixel work as your needs get more specific.

The core model is: choose an image type, load or create pixels, run ordered processing operations, then save with an explicit encoder when output behavior matters. [Image](#) is convenient when you do not need direct pixel access, while [Image<TPixel>](#) makes the in-memory pixel format part of the type so high-performance row processing and format-specific work stay explicit.

For production code, the important choices are usually not individual method names. They are whether to identify before decoding, which pixel format to use in memory, how much metadata to preserve, which encoder settings define acceptable output, and when to customize configuration for formats, memory, or security.

License

ImageSharp is licensed under the terms of the [Six Labors Split License, Version 1.0](#). See <https://sixlabors.com/pricing> for commercial licensing details.

⊗ IMPORTANT

Starting with ImageSharp 4.0.0, projects that directly depend on ImageSharp require a valid Six Labors license at build time. This enforcement applies to direct dependencies only. See [License Enforcement Changes and a New Subscription Tier](#) for details.

Install ImageSharp

ImageSharp is distributed on [NuGet](#) with preview and nightly builds available on [Feedz](#).

Package Manager

[.NET CLI](#)

[PackageReference](#)

[Paket CLI](#)

```
PM > Install-Package SixLabors.ImageSharp -Version VERSION_NUMBER
```

Prerelease versions installed via the [Visual Studio NuGet Package Manager](#) require the "include prerelease" checkbox to be checked.

How to use the license file

By default, the build searches from each project directory for `sixlabors.lic`. Place the supplied file in the directory that contains the project file, or in a subdirectory below it. Use the file as supplied; it already contains the complete license string required by the build.

⊗ IMPORTANT

Do not commit `sixlabors.lic` or a license key to public repositories such as open source projects. Use environment variables or repository secrets instead, and let contributors apply for their own independent keys at <https://licensing.sixlabors.com/>.

If you want to keep the file somewhere else, including a repository root that sits above the project directory, set `SixLaborsLicenseFile` in your project file or a shared props file:

```
<PropertyGroup>
  <SixLaborsLicenseFile>path/to/sixlabors.lic</SixLaborsLicenseFile>
</PropertyGroup>
```

If you do not want to store the license on disk, pass the license string directly from an environment variable or secret store. When extracting the value from `sixlabors.lic`, use the full file contents, not only the `key` field:

```
<PropertyGroup>
  <SixLaborsLicenseKey>$(SIXLABORS_LICENSE_KEY)</SixLaborsLicenseKey>
</PropertyGroup>
```

You can also pass the key to common .NET CLI commands.

PowerShell:

```
dotnet build -p:SixLaborsLicenseKey="$env:SIXLABORS_LICENSE_KEY"  
dotnet publish -p:SixLaborsLicenseKey="$env:SIXLABORS_LICENSE_KEY"
```

Bash and other shells that expand environment variables with `$NAME`:

```
dotnet build -p:SixLaborsLicenseKey="$SIXLABORS_LICENSE_KEY"  
dotnet publish -p:SixLaborsLicenseKey="$SIXLABORS_LICENSE_KEY"
```

Build as normal after the file or property is configured. If the license is missing or invalid, the build fails with a clear error. You do not need to reference the licensing package directly; it is carried by Six Labors libraries.

Start Here

- **Getting Started** walks through the core image types and the first end-to-end processing workflow.
- **Loading, Identifying, and Saving** covers file, stream, and buffer-based APIs plus encoder selection.
- **Working with Metadata** explains how to read and preserve EXIF, ICC, IPTC, XMP, and format-specific metadata.
- **Color Profiles and Color Conversion** covers ICC and CIECP metadata, decode-time profile handling, and explicit working-space conversion.
- **Image Formats** explains format detection, encoders, decoders, and format registration.
- **Processing Images** introduces `Mutate()` and `Clone()` pipelines.
- **Quantization, Palettes, and Dithering** explains `Quantize()`, palette-based encoders, and dithering tradeoffs.
- **Pixel Formats** and **Working with Pixel Buffers** cover direct pixel access and advanced processing.
- **Interop and Raw Memory** covers `LoadPixelData(...)`, `WrapMemory(...)`, and contiguous-buffer interop.
- **Configuration, Memory Management, and Security Considerations** cover production-focused setup.
- **Troubleshooting** covers the common failure modes around format detection, streams, memory, and disposal.
- **Migrating from System.Drawing** maps common GDI-style workflows to ImageSharp APIs.
- **Migrating from SkiaSharp** maps common SkiaSharp image workflows to ImageSharp APIs.
- **Recipes** provides copy-pasteable solutions for common tasks.

Implicit Usings

Set `UseImageSharp` in your project file to automatically import the most common ImageSharp namespaces:

```
<PropertyGroup>
  <UseImageSharp>true</UseImageSharp>
</PropertyGroup>
```

When enabled, ImageSharp adds implicit `global using` directives for:

- `SixLabors.ImageSharp`
- `SixLabors.ImageSharp.PixelFormats`
- `SixLabors.ImageSharp.Processing`

You can turn this off by removing the property or setting it to `false`.

How to Use These Docs

- Start with loading, identifying, processing, and saving if you are new to ImageSharp.
- Move to formats, metadata, color profiles, and security before accepting untrusted images in production.
- Use pixel-buffer and interop pages when you need direct memory access rather than normal processors.
- Read the migration pages when replacing APIs whose image model differs from ImageSharp's typed pixel model.

API Documentation

The API documentation is generated from the public source comments for the Six Labors libraries. Use it when you need exact type names, overloads, constructor signatures, enum values, default behavior, inherited members, and namespace-level navigation.

These reference pages are designed to sit alongside the article guides. Start with the articles when you are learning a feature or choosing an approach, then use the API reference when you need the precise member contract for implementation work.

The API reference covers the libraries documented on this site, including ImageSharp, ImageSharp.Drawing, ImageSharp.Web, Fonts, and PolygonClipper. Each product area is grouped by namespace so you can move from high-level entry points, such as image processing extensions or drawing canvas APIs, down to the supporting options, primitives, and model types.

When a reference page does not answer a design question, check the matching article section first. The source repositories remain available on [GitHub](#) for contributors and for cases where you need to inspect implementation details that are intentionally not part of the public API contract.